

[34] Redis: for vector database 총정리

개발사: Redis Labs (현 Redis)

형태: 인메모리 데이터 저장소의 벡터 검색 모듈

웹사이트: <https://redis.io/>

공개/업데이트: 2024

요약

Redis는 초고속 인메모리 데이터 구조 저장소로서, RedisVectorDB 모듈과 RediSearch의 벡터 기능을 통해 벡터 검색 능력을 갖추고 있다. Redis의 핵심 강점은 극도로 빠른 응답 시간(서브 밀리초)과 높은 처리량(초당 수백만 작업)이다. 캐싱, 세션 저장소로 널리 사용되는 Redis의 신뢰성과 생태계를 활용하면서, 벡터 검색이 필요한 실시간 애플리케이션에 완벽하게 맞다. Redis의 벡터 검색 기능은 Pinecone이나 Milvus 같은 전문 벡터 DB에 비해 배포가 간단하고, 기존 Redis 사용자들에게는 추가 인프라 없이 벡터 검색을 추가할 수 있다.

상세분석

34.1 주요 문제점과 Redis의 위치

기존 벡터 검색 솔루션의 한계:

- **초저지연 요구:** 실시간 추천, 자동완성 등에서 밀리초 단위 응답 필요, 기존 벡터 DB는 충분하지 않을 수 있음
- **인프라 복잡성:** 추가 벡터 DB 시스템 운영의 부담
- **기존 Redis 활용 미흡:** 많은 기업이 이미 Redis를 사용하고 있으나, 벡터 지원이 제한적
- **데이터 동기화:** 캐시와 벡터 DB를 별도로 운영할 때의 동기화 복잡성
- **높은 초기 비용:** 새로운 전문 시스템 도입의 학습곡선과 비용

Redis의 해결책: - 기존 Redis 인프라 위에서 벡터 검색 - 극도로 빠른 응답 - 간단한 배포와 운영 - 캐싱과 벡터 검색의 통합

34.2 핵심 벡터 검색 기능

1. RedisSearch 모듈과 벡터 지원

RedisSearch는 Redis의 검색 엔진 모듈로, 최근 버전에서 벡터 유사도 검색을 통합했다.

기본 사용법:

```
# Redis 모듈 로드
MODULE LOAD /path/to/redisearch.so

# 벡터 인덱스 생성
FT.CREATE my-index
  SCHEMA
    title TEXT
    content TEXT
    embedding VECTOR HNSW 6 TYPE FLOAT32 DIM 1536 DISTANCE_METRIC COSINE
```

2. 벡터 저장 및 관리

벡터 데이터 삽입:

```
# 벡터와 메타데이터 함께 저장
HSET doc:1 title "Article 1" content "..." embedding "binary_vector_data"

# 배치 삽입
HSET doc:2 title "Article 2" content "..." embedding "binary_vector_data"
HSET doc:3 title "Article 3" content "..." embedding "binary_vector_data"
```

Python 클라이언트 예시:

```
import redis
import numpy as np

r = redis.Redis(host='localhost', port=6379)

# 벡터 저장
vector = np.random.randn(1536).astype(np.float32).tobytes()
r.hset('doc:1', mapping={
    'title': 'Article 1',
    'content': 'Content...',
    'embedding': vector
})

# 벡터 업데이트
r.hset('doc:1', 'embedding', new_vector)

# 벡터 삭제
r.hdel('doc:1', 'embedding')
```

3. 벡터 검색

유사 벡터 검색:

```
# KNN 검색 (상위 10개 유사 벡터)
FT.SEARCH my-index "*=>[KNN 10 @embedding $query_vector]"
PARAMS 2 query_vector "binary_query_vector"
```

필터링을 포함한 검색:

```
# 특정 조건의 벡터 검색
FT.SEARCH my-index "@title:(AI OR machine-learning) =>[KNN 10 @embedding $vec]"
PARAMS 2 vec "binary_query_vector"
```

4. 거리 메트릭 지원

Redisearch 벡터 인덱싱에서 지원하는 거리 메트릭:

- **COSINE**: 코사인 거리 (정규화된 벡터에 최적)
- **L2**: 유클리드 거리 (절대 거리)
- **IP**: 내적 (dot product, 음수 허용)

5. 인덱싱 알고리즘

HNSW (Hierarchical Navigable Small World):

```

FT.CREATE index
SCHEMA
  embedding VECTOR HNSW 6
    TYPE FLOAT32
    DIM 1536
    DISTANCE_METRIC COSINE
    M 16           # 각 노드의 최대 연결 수
    EF_CONSTRUCTION 200 # 인덱싱 시 탐색 범위
    EF_RUNTIME 10    # 쿼리 시 탐색 범위

```

파라미터의 의미: - **M**: HNSW 그래프의 구조 파라미터 (클수록 정확하나 메모리 사용 증가) - **EF_CONSTRUCTION**: 인덱싱 시간의 정확도 (클수록 정확하나 느림) - **EF_RUNTIME**: 쿼리 지연시간 vs 정확도 (클수록 정확하나 느림)

34.3 Redis 아키텍처와 벡터 통합

메모리 기반 저장

```

Redis 인메모리 저장소
↓
Redisearch 모듈
↓
HNSW 벡터 인덱스 + 텍스트 인덱스
↓
메모리 상주 데이터

```

특징: - 모든 데이터가 메모리에 상주 - 극도로 빠른 접근 (나노초 단위) - 영구 저장은 RDB/AOF로 선택적 처리

성능 최적화

SIMD 최적화: - 벡터 거리 계산에 CPU SIMD 명령어 활용 - 배치 처리로 캐시 효율성 향상

메모리 압축: - FLOAT32 (4바이트/차원) 지원으로 메모리 절감 - 예: 1536차원 벡터 = 6KB

병렬 처리: - 여러 검색 스레드 지원 - Redis 6.0 이후 멀티 스레드 I/O

34.4 배포 모델

1. 단일 노드 Redis

```
클라이언트
↓
Redis 서버 (벡터 인덱스 + 데이터)
↓
메모리/디스크 (RDB/AOF)
```

사용: 개발, 소규모 프로덕션

2. Redis Cluster

```
클라이언트 (클러스터 클라이언트)
↓
Redis 마스터 노드들 (샤딩)
↓
각 노드: 벡터 인덱스의 부분집합
```

사용: 대규모 프로덕션, 높은 가용성

샤딩 전략: - 문서 ID 기반 해시 샤딩 - 각 샤드는 독립적인 벡터 인덱스 유지 - 검색 시 모든 샤드에 병렬 쿼리

3. Redis Stack

Redis Labs의 통합 솔루션으로, RedisSearch, RedisJSON, RedisTimeSeries 등을 포함.

34.5 실제 사용 사례

1. 실시간 추천 시스템:

```

import redis
from redis.commands.search.query import Query

r = redis.Redis(host='localhost', port=6379, decode_responses=True)

# 사용자 선호도 벡터
user_vector = get_user_embedding(user_id)

# 상품 추천 (상위 5개)
result = r.ft('product-index').search(
    Query(f"*=>[KNN 5 @embedding $vec]")
    .params(vec=user_vector)
)

recommendations = [doc['product_id'] for doc in result.docs]

```

2. 자동완성 + 의미 검색:

```

# 검색어 임베딩
query_text = "machine learning"
query_vector = embed_text(query_text)

# 텍스트 매칭 + 벡터 유사도
results = r.ft('article-index').search(
    Query("@title:machine* =>[KNN 10 @embedding $vec]")
    .params(vec=query_vector)
)

```

3. 실시간 모니터링/이상 탐지:

```

# 새로운 이벤트 벡터와 유사한 과거 이벤트 검색 (이상 패턴)
new_event_vector = extract_features(event)

similar_events = r.ft('event-index').search(
    Query(f"@severity:high =>[KNN 20 @embedding $vec]")
    .params(vec=new_event_vector)
)

# 유사한 이상 사건이 많으면 알림
if len(similar_events.docs) > 10:
    send_alert()

```

34.6 성능 특성

검색 성능

지연시간: - 단일 KNN 쿼리: 0.1-1ms (수백만 벡터) - 필터링 포함: 1-10ms (필터 선택도에 따라)

처리량: - 초당 수백만 벡터 검색 (클라이언트 연결 수에 따라)

메모리 사용

벡터 저장: - FLOAT32: 차원 × 4바이트 - 1536차원: 6KB/벡터

인덱싱 오버헤드: - HNSW 메타데이터: 원본의 약 50-100% - 1백만 벡터: 약 1-2GB 추가 메모리

확장성

단일 노드 제약: - 메모리 크기가 최대치 (일반적으로 256GB) - 약 4천만-4억 벡터 저장 가능 (메모리에 따라)

클러스터 확장: - 16개 샤드로 확장하면 선형 확장 - 초당 수십억 쿼리 처리 가능

34.7 본 논문과의 관계

Exqutor은 텍스트 쿼리를 벡터 기반 검색으로 변환하는 하이브리드 검색 시스템이다. Redis는 다음의 측면에서 Exqutor과 관련:

1. **초저지연 요구:** Exqutor이 실시간 응답을 요구하는 환경(예: 검색 자동완성)에서는 Redis의 극도로 빠른 성능이 이상적
2. **기존 인프라 활용:** 많은 애플리케이션이 이미 Redis를 사용 중이므로, Exqutor을 Redis 위에 구축하면 배포 단순화
3. **캐싱 통합:** Exqutor의 검색 결과 캐싱과 벡터 인덱싱을 하나의 Redis에서 관리
4. **클러스터 확장:** Exqutor이 대규모로 배포될 때 Redis Cluster를 통한 수평 확장 가능
5. **간단한 운영:** Redis의 광범위한 도구와 모니터링 생태계 활용

추가 제기 문제

1. **메모리 제약:** 인메모리 저장소의 근본적 한계를 어떻게 극복할 것인가? 디스크 기반 벡터 저장이 성능에 미치는 영향은?
2. **정확도 vs 성능:** HNSW의 `EF_RUNTIME` 파라미터를 어떻게 동적으로 조정할 것인가? 쿼리별 요구 정확도에 따른 자동 조정?
3. **클러스터에서의 정확도:** 벡터 인덱스를 여러 샤드에 분산할 때, 전역 KNN 결과의 정확도 보장은?
4. **메타데이터 필터링의 성능:** 복잡한 필터 조건과 벡터 검색의 결합 시 성능 저하 정도는?

5. **동적 업데이트**: 높은 빈도의 벡터 추가/삭제 중에도 검색 성능을 유지할 수 있는가?
6. **다중 벡터 필드**: 문서당 여러 벡터 필드(제목, 내용, 카테고리별 임베딩)를 지원할 때의 인덱싱 전략은?
7. **Redis vs 전문 벡터 DB**: Redis가 어떤 상황에서 Pinecone이나 Milvus를 대체할 수 있는가? 트레이드오프는?